Directory Operations

1 Basic Idea

Definition

Directory operations are system calls used to create, remove, open, read, rename, and link directory entries.

- Directory system calls vary more across operating systems than ordinary file calls.
- UNIX provides a useful model for understanding common directory operations.
- Directories are special files, but they should not be treated like ordinary byte streams by user programs.
- Directory operations manage names and links between names and files.

2 Common Directory Operations

Operation	Meaning
Create	Create a new empty directory.
Delete	Remove an empty directory.
Opendir	Open a directory so its entries can be read.
Closedir	Close an open directory and free internal table space.
Readdir	Return the next entry from an open directory.
Rename	Change the name of a directory.
Link	Add another directory entry for an existing file.
Unlink	Remove a directory entry.

3 Create

Create Directory

Creating a directory makes a new directory that is empty except for `.` and `..`.

- The new directory initially contains no user files.
- The special entry `.` names the directory itself.
- The special entry `..` names the parent directory.
- These entries are usually inserted automatically by the system.
- In a few systems, the `mkdir` program may create them.

4 Delete

Delete Directory

Deleting a directory removes it from the file system, but normally only if it is empty.

- A directory can be deleted only when it is empty.
- A directory containing only `.` and `..` is considered empty.
- The entries `.` and `..` cannot be deleted as ordinary entries.
- Requiring emptiness prevents accidental deletion of a whole subtree.

Safety Rule

Deleting a nonempty directory is normally rejected because it would otherwise remove names for files and subdirectories below it.

5 Opendir and Closedir

Reading Directories

A directory must be opened before a program can read its entries.

- A listing program opens a directory to read the names it contains.
- Opening a directory is analogous to opening a file before reading it.
- After the directory has been read, it should be closed.
- Closing frees internal operating-system table space.

Call	Purpose
`opendir`	Prepare a directory for entry-by-entry reading.
`closedir`	Finish directory reading and release OS resources.

6 Readdir

Readdir

`readdir` returns the next entry in an open directory using a standard format.

- Directory entries can be read one at a time.
- Older systems sometimes allowed ordinary `read` on directories.
- That forced programmers to understand the internal directory format.

- `readdir` hides the internal directory representation.
- It returns directory entries in a standard form.
- This works even if file systems use different internal directory structures.

Main Benefit

`readdir` gives programs a portable way to read directory entries without depending on raw directory layout.

7 Rename

Rename Directory

Directories can often be renamed much like ordinary files.

- Rename changes a directory's name.
- It does not require copying all files inside the directory.
- The directory entry is updated.
- The actual directory contents remain in place.

8 Link

Hard Link

A **hard link** creates another directory entry that refers to the same existing file.

- Linking lets the same file appear in more than one directory.
- The system call specifies an existing file and a new path name.
- The new path becomes another name for the same file.
- The file's i-node link count is incremented.
- The link count tracks how many directory entries refer to the file.
- This kind of link is called a hard link.

9 Unlink

Unlink

Unlink removes a directory entry.

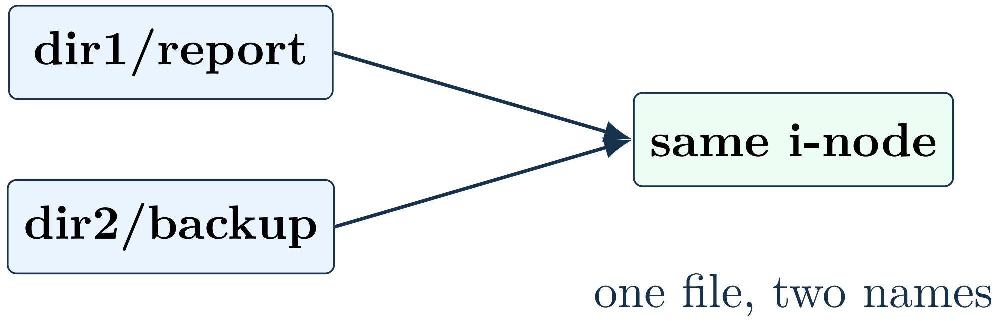
- If the file has only one directory entry, unlinking removes it from the file system.

- If the file has multiple hard links, only the specified path name is removed.
- Other path names for the same file remain valid.
- In UNIX, the system call used to delete ordinary files is actually `unlink`.

Link Count Rule

A file's storage can be freed only after its last directory entry has been unlinked.

10 Hard Link Example



11 Symbolic Links

Symbolic Link

A **symbolic link** is a special file that contains the path name of another file.

- A symbolic link does not point directly to the same internal file structure.
- Instead, it names another file by path.
- When the symbolic link is opened, the file system reads the stored path.
- Then lookup starts again using that path.
- Symbolic links can cross disk boundaries.
- They can also refer to files on remote computers.
- They are usually less efficient than hard links.

12 Hard Links vs. Symbolic Links

Feature	Hard link	Symbolic link
Target	Same internal file object or i-node.	A path name stored in a small special file.
Lookup	No second path lookup is needed.	File system follows the stored path and restarts lookup.
Disk boundary	Usually cannot cross file-system boundaries.	Can cross disk and network boundaries.
Efficiency	More efficient.	Less efficient because path lookup is repeated.
Failure case	Remains valid while the underlying file still has links.	Can become dangling if the target path disappears.

13 Operation Groups

Group	Operations
Lifecycle	Create, delete.
Traversal	Opendir, readdir, closedir.
Naming	Rename.
Multiple names	Link, unlink.
Protection	Other system-specific calls may manage directory permissions.

Final Point

Directory operations manage names, directory contents, and links. Hard links add another name for the same file; symbolic links store a path to another file.